

RAPPORT D'ARCHITECTURE & DE SPÉCIFICATION DU SYSTÈME BACKEND

*Plateforme d'Administration, de Cartographie Réseau et de Gestion Globale des
Réclamations*

Candidat : Amine

Rôle Système : Administrateur de Base de Données (okasu)

Cadre Académique : Projet de Fin d'Études (PFE)

Environnement Target : Windows 11 (Lenovo Core Architecture)

Moteur relationnel : PostgreSQL Database Engine

Framework : Django REST Framework (DRF) v3.x & Django 5.x

1. INTRODUCTION FONDAMENTALE ET VISION DE L'ARCHITECTURE

Ce document constitue le référentiel technique exhaustif de l'infrastructure logicielle backend développée dans le cadre de notre Projet de Fin d'Études (PFE). La solution conçoit et implémente un système centralisé de supervision, de cartographie topologique et de gestion critique des réclamations et pannes pour un écosystème d'infrastructures réseau.

Pour assurer une séparation stricte des responsabilités, une extensibilité maximale et une isolation des composants applicatifs, l'architecture retient le paradigme du ****Découplage Backend/Frontend****. Le backend fait office de fournisseur d'API REST hautement sécurisé et optimisé, matérialisé par le framework ****Django REST Framework (DRF)****. Le frontend, quant à lui, consomme ces APIs de manière asynchrone pour piloter l'interface utilisateur.

1.1 Le Patron d'Architecture MVC / MVT Élargi

Django s'appuie nativement sur l'architecture ****MVT (Model-View-Template)****. Toutefois, dans le cadre d'un développement d'API REST découplée, la couche *Template* (HTML) est entièrement substituée par une couche de ****Sérialisation (Serializers)**** fournie par DRF. L'architecture se décline ainsi :

- **Modèles (Models)** : Représentent la couche d'abstraction de données (ORM). Ils encapsulent les structures relationnelles PostgreSQL, les contraintes d'intégrité, et la logique métier de persistance.
- **Sérialiseurs (Serializers)** : Agissent comme des traducteurs bidirectionnels avancés. Ils convertissent les instances de modèles complexes en formats d'échange standards (JSON) pour les requêtes sortantes, et valident les données entrantes avant de les sérialiser en objets Python compréhensibles par la base de données.
- **Vues (Views / API Views)** : Constituent les contrôleurs de l'application. Elles interceptent les requêtes HTTP, valident les droits d'accès via les politiques d'authentification, orchestrent les appels à l'ORM, et retournent des réponses normalisées.

1.2 Découpage Modulaire du Projet (Analyse Structurelle)

Comme illustré dans l'arborescence physique du projet, le code est segmenté en sous-modules applicatifs étanches, chacun répondant à un domaine métier précis :

APPLICATION (MODULE)	RESPONSABILITÉ FONCTIONNELLE PRINCIPALE	TECHNOLOGIES INTERNES CLÉS
config	Noyau de configuration global du projet. Centralise les configurations de base de données, middlewares, routage principal, et variables d'environnement.	settings.py , urls.py , WSGI/ASGI settings
accounts		

APPLICATION (MODULE)	RESPONSABILITÉ FONCTIONNELLE PRINCIPALE	TECHNOLOGIES INTERNES CLÉS
	Gestion du cycle de vie des utilisateurs, authentification robuste et implémentation du contrôle d'accès basé sur les rôles (RBAC).	AbstractUser, SimpleJWT tokens, permissions DRF
sites_reseau	Référentiel géospatial et technique des infrastructures. Modélise les équipements topologiques et leurs états opérationnels.	Django ORM Models, Statut Choices (UP/DOWN)
reclamations	Moteur transactionnel des tickets d'incidents, historisation des pannes et fil conducteur de communication (commentaires de suivi).	Clés étrangères multiples, validation d'états, signaux ORM
dashboard	Module analytique décisionnel. Centralise les indicateurs clés de performance (KPI) et pré-calcule les structures de graphes complexes.	SQL aggregates, Q objects, TruncDay functions
cartographie	Moteur de rendu spatial assurant la liaison visuelle et géographique entre l'état des pannes et les coordonnées physiques des sites.	Spatial/JSON payload formatters

2. LA COUCHE DE PERSISTANCE : BASE DE DONNÉES

POSTGRESQL ET ORM

La persistance de l'application repose sur le système de gestion de base de données relationnelle (SGBDR) **PostgreSQL**, accédé par le profil utilisateur dédié `okasu`. L'accès à la base s'effectue exclusivement au travers de l'**ORM (Object-Relational Mapping)** de Django, éliminant l'écriture manuelle de requêtes SQL brutes sujettes aux failles d'injection, tout en garantissant une portabilité et une abstraction de haut niveau.

2.1 Les Dépendances Fondamentales et Bibliothèques Tierces (requirements.txt)

Le fichier `requirements.txt` compile les modules Python tiers indispensables à l'exécution du serveur :

- `django` : Le framework applicatif global.
- `djangorestframework` : L'extension logicielle permettant de structurer le projet sous forme d'API REST.
- `djangorestframework-simplejwt` : Bibliothèque d'authentification basée sur les jetons Web JSON (JWT).
- `psycopg2-binary` : Le pilote d'accès natif (driver) permettant à Python et à Django de communiquer de façon performante avec le serveur PostgreSQL via des sockets bas niveau.

2.2 Le Mécanisme de Migration : De Python vers SQL

L'ORM utilise un système binaire de synchronisation de schéma appelé **Migrations**. Lorsque nous modifions un fichier `models.py`, l'opération se découpe en deux phases :

```
python manage.py makemigrations
```

Cette commande analyse les fichiers `models.py`, les compare à l'état précédent, et génère un fichier de transition en Python pur (situé dans le dossier `migrations/` de chaque application). Ce fichier décrit les modifications structurelles (ex: *Add field*, *Create model*).

```
python manage.py migrate
```

Cette commande lit les fichiers de migration non encore appliqués et traduit les instructions Python en requêtes de définition de données SQL (ex: `CREATE TABLE`, `ALTER TABLE`) exécutées directement au sein de PostgreSQL.

Note de Soutenance Importante : Pendant la phase de développement sur l'environnement Windows 11, l'erreur `ECONNREFUSED 127.0.0.1:8000` rencontrée sous Postman signifie simplement que le serveur HTTP de développement Django (interfacé via `manage.py runserver`) s'est arrêté, rompant la liaison socket locale. Elle ne traduit pas une corruption du code ou de la base PostgreSQL.

3. MODULE D'AUTHENTIFICATION AVANCÉE ET SÉCURITÉ

RBAC (ACCOUNTS)

La sécurité globale repose sur deux piliers majeurs : l'authentification par jeton à durée de vie limitée (**JWT**) et l'autorisation par contrôle d'accès basé sur les rôles (**RBAC**).

3.1 Le Modèle Utilisateur Personnalisé et l'Énumération des Rôles

Pour dépasser les limitations du modèle utilisateur par défaut de Django, une extension de la classe `AbstractUser` est implémentée dans l'application `accounts`. Elle intègre un champ typé `role` adossé à une classe de choix énumérés :

```
# accounts/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models

class Role(models.TextChoices):
    ADMIN = 'ADMIN', 'Administrateur'
    CALL_CENTER = 'CALL_CENTER', 'Agent Call Center'
    INGENIEUR = 'INGENIEUR', 'Ingénieur Réseau'

class Utilisateur(AbstractUser):
    role = models.CharField(
        max_index=20,
        choices=Role.choices,
        default=Role.INGENIEUR
    )
```

3.2 Le Protocole JWT (JSON Web Token)

Contrairement à l'authentification par session traditionnelle qui stocke l'état de l'utilisateur en mémoire serveur, l'authentification JWT est **stateless** (sans état). Le protocole se déroule selon une séquence rigoureuse :

1. Le client (Postman ou Frontend) transmet ses identifiants en JSON via une requête `POST` sur l'endpoint dédié `/api/token/`.
2. Le backend valide les hashes de mots de passe. Si l'authentification réussit, la bibliothèque `SimpleJWT` génère un couple de jetons cryptographiques signés numériquement :
 - **Access Token** : Valide pour une courte période (ex: 5 minutes). Il encapsule l'identité et le rôle de l'utilisateur. Chaque requête ultérieure vers une API protégée doit inclure ce token dans l'entête HTTP sous la forme `Authorization: Bearer <token>`.

- **Refresh Token** : Valide pour une plus longue période (ex: 24 heures). Il sert exclusivement à demander un nouvel Access Token sur l'endpoint `/api/token/refresh/` sans forcer l'utilisateur à se réauthentifier avec ses identifiants.

Défense face au Jury : En cas de réception d'une réponse `401 Unauthorized` avec le message *"Informations d'authentification non fournies"*, j'explique que la politique globale de sécurité de DRF a intercepté la requête, car l'Access Token a expiré ou l'entête `Bearer` était syntaxiquement malformée lors du transfert HTTP.

4. SPÉCIFICATION TECHNIQUE DU MODULE MÉTIER (RECLAMATIONS & SITES_RESEAU)

Le module `reclamations` orchestre la gestion opérationnelle des incidents survenus sur les infrastructures modélisées par l'application `sites_reseau`.

4.1 Structure du Fichier Central des Vues et Sécurité des Routes

Le fichier `reclamations/views.py` expose l'intégralité du cycle de vie des tickets d'incidents (CRUD). L'implémentation met en œuvre le décorateur de fonction `@api_view` combiné aux classes de permissions DRF :

4.2 L'Architecture du Code Métier Optimisé

```
# reclamations/views.py
from rest_framework import status
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from .models import Reclamation
from .serializers import ReclamationSerializer, CommentaireSerializer
from accounts.models import Role

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def liste_reclamations(request):
    statut_filter = request.query_params.get('statut')
    # OPTIMISATION AVANCÉE : Élimination du problème N+1
    reclamations = Reclamation.objects.select_related('site_reseau', 'cree_par').all()
    if statut_filter:
        reclamations = reclamations.filter(statut=statut_filter)
    serializer = ReclamationSerializer(reclamations, many=True)
    return Response(serializer.data)

@api_view(['POST'])
@permission_classes([IsAuthenticated])
def creer_reclamation(request):
    if request.user.role not in [Role.ADMIN, Role.CALL_CENTER]:
        return Response({'error': 'Permission refusée.'}, status=status.HTTP_403_FORBIDDEN)
    serializer = ReclamationSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save(cree_par=request.user)
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

4.3 Résolution Technique Avancée : Le Problème SQL N+1

Le concept d'optimisation majeur intégré dans la fonction `liste_reclamations` réside dans l'appel à la méthode `select_related()` de l'ORM.

****Le Phénomène N+1 (Sans Optimisation) :**** Si la table des réclamations contient N enregistrements, et que le sérialiseur doit afficher pour chaque ticket le nom du site réseau associé et le nom de l'utilisateur créateur, l'ORM exécuterait naïvement 1 requête initiale pour l'obtention de la liste globale, suivie de N requêtes SQL secondaires de jointure à l'unité. Pour 100 tickets, 101 requêtes frapperaient PostgreSQL, dégradant les temps de réponse de manière exponentielle.

****La Solution Apportée :**** En spécifiant `select_related('site_reseau', 'cree_par')`, l'ORM Django génère en amont une requête SQL unique incorporant des clauses de jointures relationnelles directes (`INNER JOIN`). Le serveur PostgreSQL résout l'intégrité en un seul passage, ramenant la complexité de traitement à une seule et unique requête, optimisant les entrées/sorties (I/O) de manière drastique.

5. ALGORITHMIQUE AVANCÉE DU MODULE DÉCISIONNEL (DASHBOARD)

L'application `dashboard` constitue le sommet de la logique d'optimisation de notre projet de fin d'études. Elle fournit une vue analytique consolidée, filtrable dans le temps, tout en minimisant la charge sur le serveur PostgreSQL.

5.1 Analyse du Code Source Algorithmique

La vue principale implémente un traitement en flux unique pour agréger les métriques des sites et des réclamations :

```
# dashboard/views.py
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from django.db.models import Count, Q
from django.db.models.functions import TruncDay
from django.utils import timezone
from datetime import timedelta
from reclamations.models import Reclamation
from sites_reseau.models import SiteReseau

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def statistiques(request):
    # 1. Traitement du paramètre de filtrage temporel dynamiquement
    jours_param = request.query_params.get('jours', '30')
    try:
        nb_jours = int(jours_param)
    except ValueError:
        nb_jours = 30
    date_limite = timezone.now() - timedelta(days=nb_jours)

    # 2. Agrégation en base en une seule requête pour les sites réseau
    stats_sites = SiteReseau.objects.aggregate(
        total=Count('id'),
        up=Count('id', filter=Q(statut='UP')),
        down=Count('id', filter=Q(statut='DOWN')),
        degrade=Count('id', filter=Q(statut='DEGRADE')),
        perturbe=Count('id', filter=Q(statut='PERTURBE'))
    )

    # 3. Agrégation des tickets filtrés temporellement
    stats_tickets = Reclamation.objects.filter(created_at__gte=date_limite).aggregate(
```

```

        total=Count('id'),
        ouverts=Count('id', filter=Q(statut='ouvert')),
        en_cours=Count('id', filter=Q(statut='en_cours')),
        resolu=Count('id', filter=Q(statut='resolu'))
    )

    total_t = stats_tickets['total']
    resolu_t = stats_tickets['resolu']
    taux = round((resolu_t / total_t) * 100, 2) if total_t > 0 else 0

    # 4. Extraction du Top 5 des sites les plus touchés
    top_sites = SiteReseau.objects.annotate(
        num_reclamations=Count('reclamations',
filter=Q(reclamations__created_at__gte=date_limite))
    ).order_by('-num_reclamations')[:5].values('id', 'codeSite', 'nom', 'num_reclamations')

    # 5. Groupement chronologique pour le Line Chart frontend
    evolution = list(Reclamation.objects.filter(created_at__gte=date_limite)
        .annotate(jour=TruncDay('created_at')).values('jour')
        .annotate(total=Count('id')).order_by('jour'))

    return Response({
        'periode_filtree_en_jours': nb_jours,
        'sites': stats_sites,
        'tickets': {
            'total': total_t, 'ouverts': stats_tickets['ouverts'],
            'en_cours': stats_tickets['en_cours'], 'resolu': resolu_t,
            'taux_resolution': f"{taux}%"
        },
        'graphiques': {'top_sites_impactes': list(top_sites), 'evolution_tickets':
evolution}
    })

```

5.2 Les Concepts Mathématiques et SQL de l'Agrégation

Pour garantir l'immédiateté du Dashboard, la logique logicielle s'appuie sur deux mécanismes d'agrégation haut de gamme :

- **L'Agrégation Conditionnelle (Conditional Aggregation)** : L'utilisation combinée de `Count()` et de l'objet de requête complexe `Q()` permet de générer des requêtes SQL intégrant des clauses de filtrage sélectives directement au sein des fonctions d'agrégation :

```
COUNT(id) FILTER (WHERE statut = 'UP')
```

Au lieu d'exécuter cinq requêtes de comptage différentes (une pour le total, une pour les UP, une pour les DOWN, etc.), l'ORM compacte l'ensemble dans une unique instruction de sélection. PostgreSQL parcourt la table des sites une seule fois (*Single Table Scan*) et incrémente les compteurs adéquats à la volée.

- **La Troncature Temporelle (TruncDay) :** Pour alimenter l'affichage temporel (Line Chart), la fonction `TruncDay('created_at')` intercepte les horodatages précis (ex: `2026-05-17 23:45:12`) et tronque la composante horaire pour la ramener au jour brut (`2026-05-17 00:00:00`). Elle applique ensuite un modificateur SQL `GROUP BY` sur cette date normalisée pour totaliser le volume d'incidents par jour.

6. FICHES DE SYNTHÈSE DIDACTIQUES POUR L'ÉPREUVE DE SOUTENANCE

Cette section compile les questions pièges potentielles des membres du jury et formule les réponses techniques, méthodologiques et architecturales à adopter pour valoriser le travail accompli.

6.1 Formulaire de Réponses aux Questions Clés du Jury

Question 1 : "Pourquoi avoir choisi une architecture découplée plutôt que d'utiliser les templates HTML natifs de Django ?"

Stratégie de réponse : L'architecture découplée externalise entièrement la couche de présentation (Frontend). Cela offre trois bénéfices majeurs : une expérience utilisateur dynamique et fluide (Single Page Application), une spécialisation des compétences de développement, et surtout, l'évolutivité. Notre backend Django DRF fait office d'API universelle : aujourd'hui il alimente un tableau de bord web, demain il pourra sans aucune modification logicielle de sa structure alimenter une application mobile Android ou iOS.

Question 2 : "Expliquez l'intérêt concret de l'utilisation des objets Q et de la fonction aggregate() dans votre dashboard."

Stratégie de réponse : L'approche naïve aurait consisté à charger les données en mémoire Python et à effectuer des boucles de filtrage ou à multiplier les requêtes SQL unitaire avec des `.filter().count()`. Cela s'avère catastrophique en production. En mobilisant `aggregate()` couplé à `Q()`, nous déportons l'intégralité du calcul algorithmique directement sur le moteur de base de données PostgreSQL qui est hautement optimisé pour l'indexation et le requêtage de masse. Le serveur web ne fait que réceptionner le résultat calculé final.

Question 3 : "Quelle est la différence fondamentale entre les instructions de sérialisation et le modèle ORM ?"

Stratégie de réponse : L'ORM (Modèles) est responsable du dialogue vertical entre le code Python et la base de données relationnelle PostgreSQL. Il manipule des objets complexes typés. Le Sérialiseur, quant à lui, est responsable du dialogue horizontal entre notre API et l'extérieur (le réseau). Il traduit ces instances d'objets complexes en flux textuels standardisés JSON compressés et sécurisés pour l'échange de données.

6.2 Synthèse du Flux Applicatif Frontend vers Backend

Pour clore la présentation du backend, voici la chronologie exacte de l'échange réseau qui se produira lorsque l'interface utilisateur affichera l'écran des statistiques :

1. L'utilisateur frontend clique sur le menu "Dashboard" et sélectionne le filtre "7 derniers jours".

2. Le moteur JavaScript frontend extrait l'Access Token stocké localement et émet de manière asynchrone une requête HTTP dotée du verbe `GET` vers l'adresse résolue : `http://127.0.0.1:8000/api/dashboard/stats/?jours=7`.
3. Le routeur global de Django intercepte l'URI, valide la validité de la signature cryptographique du jeton JWT via le middleware d'authentification de DRF, et transfère le flux à la fonction contrôleur `statistiques(request)` du module `dashboard`.
4. La vue extrait le Query Parameter `jours`, calcule la date pivot par soustraction temporelle, et soumet les requêtes d'agrégation optimisées à PostgreSQL.
5. PostgreSQL résout les fonctions de comptage conditionnelles et les jointures en un temps record grâce aux index de clés primaires et retourne les lignes brutes.
6. La vue Django DRF enveloppe ces dictionnaires de données dans un objet `Response` qui génère un flux de données JSON structuré en blocs distincts (*sites*, *tickets*, *graphiques*), envoyé sur le réseau avec un code de succès `200 OK`.

— FIN DU RAPPORT DE SPÉCIFICATION TECHNIQUE DU BACKEND —